CONCURRENCY, PROCESSES, THREADS, CORES, AND SCHEDULING – THE COMPLETE PICTURE

A detailed visual guide for C / Linux beginners

LEGEND: Program · Process · Thread · Scheduler / Run Queue · CPU Core / Hardware · Shared Memory · Private Thread State · Problems / Hazards · Solutions / Synchronization · Waiting / Blocked

1 THE BIG PIPELINE

Program file (Executable) (.c / source) → Compile (gcc -o app app.c) → Executable (binary) (./app) → Execute / Run (OS loader) → Process created by OS (PID 1234) → Process contains one or more threads (T1 T2 ... Tn) → Runnable threads go to Scheduler / Run Queue → Runnable rastains go to Scheduler / Run Queue → Scheduler assigns runnable threads to CPU cores → CPU cores execute instructions

A process always has at least one thread: the main thread.

2 WHAT EACH THING IS

- **Program** — Code / executable on disk, ready to run.
- **Process** — Running instance of a program. OS-managed resource container. Owns address space, heap, global vars, files (FDs), etc.
- **Thread** — Execution path inside a process. Has its own stack, registers, instruction pointer, thread ID.
- **Core** — Physical CPU execution unit that executes instructions.

CONCURRENCY vs PARALLELISM

Concurrency = Multiple tasks making progress overlapping in time.
Parallelism = Tasks literally executing at the SAME TIME on different CPU cores.

1-CORE (Concurrency, not parallelism)

T1 T2 T3 T4 T6

Tasks interleave on one core.

MULTI-CORE (Parallelism)

Core 1	T1	T2	T3	T3
Core 2	T2	T3	T2	T4

Tasks execute at the same time on different cores.

3 WHAT IS SHARED VS PRIVATE

One Process (PID 1234)

Thread 1 (TID 101) · Thread 2 (TID 102) · Thread 3 (TID 103)

SHARED IN THE PROCESS (All threads share)

- Code / Text (program instructions)
- Global variables & Static data
- Heap (dynamic memory)
- Open Files / File descriptors
- Environment variables
- Other process resources (signals, cwd, etc.)

PRIVATE TO EACH THREAD

- Stack (function calls, local vars)
- Registers (PC, SP, general)
- Thread ID (TID)
- Stack Pointer / Frame Pointer
- Signal Mask
- Thread-Local Storage (TLS)
- Current execution state

Threads share the process heap.
Threads do NOT each get their own heap.

4 PROCESSES VS THREADS

TWO SEPARATE PROCESSES

(From the same program)

Process 1001 — PID 1001 (T1 only) — Address Space: Heap, Globals, Files, Env

Process 1002 — PID 1002 (T1 only) — Address Space: Heap, Globals, Files, Env

Separate address spaces, heaps, globals, resources. Isolated by default.

ONE PROCESS WITH MULTIPLE THREADS

(PID 2000)

Process 2000: Main Thread · Worker 1 · Worker 2 · Logger Thread

Shared address space, heap, globals, files, etc.

A thread belongs to exactly one process.
Two processes do not share the same threads.

Processes communicate with IPC: Pipe · Socket · Shared Memory · Msg Queue · Files

Threads communicate through shared memory / variables: Global / Static / Shared data

5 SCHEDULER, RUN QUEUE & CONTEXT SWITCH

RUN QUEUE

Runnable threads waiting for CPU time

T1 T3 T7 T9 ... Tn

A run queue contains threads that are ready to run but blocked due to waiting on I/O, locks, etc.

The scheduler picks runnable threads and dispatches them to available cores.

Runnable / Ready ↔ Blocked / Waiting (e.g., I/O, lock, sleep)

CONTEXT SWITCH — STEP BY STEP

1. Save state of current thread
2. Decide next thread to (Scheduler)
3. Save T1, load T2 context (Registers, PC, SP, etc.)
4. Load T2 context into CPU (Restores state)
5. Continue execution of new thread

A context switch does not restart a thread; it resumes where it stopped.

5 CORES, THREADS, RUN QUEUES, AND THE SCHEDULER

THREAD STATES & TRANSITIONS

Running (Currently executing on a CPU core) → Runnable / Ready (Waiting for CPU time slice, preemption) → Blocked / Waiting (Waiting for I/O, lock, I/O, lock, sleep)

Terminated (Finished execution)

Threads move between states.

RUN QUEUE

(per CPU core)

T1 T3 T5 T7 ... Tn

Runnable threads waiting for CPU time.

CPU CORES (Hardware)

Core 1 T1 · Core 2 T3 · Core 3 T6 · Core 4 T2

6 WHAT CONCURRENCY REALLY MEANS, AND WHY IT NEEDS SOLUTIONS

WHAT IS CONCURRENCY?

Concurrency means multiple tasks exist around the same time or interleaving progress of tasks. They may share data and resources (memory, files, I/O devices).

Because of order & timing unpredictability, bad things happen.

Concurrency itself is not the problem — uncontrolled shared access to data is the problem.

WHY DOES CONCURRENCY NEED SOLUTIONS?

- The exact order of thread execution is not predictable.
- Threads can access the same memory at the same time.
- One thread may read half-updated data from another.
- Threads can block each other forever.
- Too much coordination can hurt performance.

The goal of synchronization:
Make shared access SAFE, COORDINATED, and VISIBLE in the CORRECT ORDER.

7 MAIN CONCURRENCY PROBLEMS

	PROBLEM	WHAT IT IS	WHAT IT HAPPENS	WHAT GOES WRONG	EXAMPLE FIX
1	Race condition	Unsynchronized concurrent access to shared data	Threads access same data at the same time	Lost updates, corrupted data, inconsistent state	Mutex, atomic operations, locks, critical sections
2	Deadlock	Each thread waits forever for another	Circular wait, hold & wait	Program hangs	Lock ordering, timeouts, try-locks, avoid cycles
3	Starvation	A thread gets no CPU time or resources	No fairness, others always win	Thread never runs, starves	Fair scheduling, priorities, round-robin
4	Livelock	Threads keep reacting to each other, no progress	Quickly retry operations, conflicting actions	High CPU use, no progress	Backoff, random delays, change strategy
5	Visibility / Ordering	Writes not visible to other threads in order	CPU caches, compiler reordering, weak memory	Stale reads, incorrect behavior	Memory barriers, atomics, volatile, fences
6	Lost updates	One thread overwrites another's update	Read-modify-write race	Data trash, incorrect results	Locks, atomic RMW, transactions
7	False sharing	Threads modify different vars on same cache line	Invalidation ping-pong	Poor performance, scalability loss	Padding, align data, reduce sharing
8	Priority inversion	Low-priority thread blocks high-priority thread	Mutex held by low-priority thread	High priority waits unnecessarily	Priority inheritance, ceiling protocol
9	Lost wakeup / Missed signal	Signal/condition sent before thread waits	Bad timing of signal vs wait	Thread waits forever ("sleep forever")	Use condition variables correctly

8 GENERAL SOLUTIONS AND WHEN TO USE THEM

SYNCHRONIZATION TOOL BOX (Use the right tool for the right job)

- Mutex – Protect critical sections (mutual exclusion)
- Read/Write Lock – Many readers or one writer
- Semaphore – Counted resource control
- Condition Variable – Wait & signal with conditions
- Atomic Operations – Lock-free counters / flags / updates
- Memory Barriers / Fences – Ensure visibility & ordering
- Spinlock – Busy-wait for very short critical sections
- Thread-Local Storage (TLS) – Per-thread data
- Message Queue / Channel – Thread/Process communication
- Pipe / Socket – IPC (inter-process or inter-thread)
- Barrier – Coordinate phases among threads
- Lock-Free Algorithms – High scalability, low contention
- Good Design – Minimize shared state, avoid unnecessary sharing

CHOOSE THE RIGHT TOOL (Quick Guide)

Need exclusive access	Mutex
Need to wait for event / condition	Condition Variable
Need count / resource control	Semaphore
Need simple counter / flag	Atomics
Need many readers / few writers	Read/Write Lock
Need inter-thread communication	Queues / Channels

Use the simplest tool that solves the problem correctly and efficiently.

9 PRACTICAL EXAMPLES

A. RACE CONDITION (Bank Account)

Thread A (Deposit)	Thread B (Withdraw)
1. Read balance (1000)	1. Read balance (1000)
2. Compute (1000 + 500)	2. Compute (1000 - 200)
3. Write balance (1500)	3. Write balance (800)

Final result = 800 ✗
Should be 1300 ✓

Fix: Use mutex / atomic or serializable operations

B. DEADLOCK (Two Locks)

Thread 1: Lock A, then Lock B
Thread 2: Lock B, then Lock A

Deadlock: Both wait forever.

Fix: Enforce lock ordering.

C. PRODUCER – CONSUMER (using Condition Variable)

Producer Thread: Add items to queue, Signal not empty
Consumer Thread: Wait (queue not empty), Remove item
Queue (Buffer)

Fix: Mutex + condition variable
Correct signaling & while loops.

10 THREADS VS PROCESSES: WHEN TO USE WHICH

USE THREADS WHEN

- Tasks share data frequently
- Need high performance / low latency
- Create thousands of concurrent tasks
- Lightweight tasks (e.g., quick I/O)
- Same process resources (heap, globals)
- Want cheaper context switches

USE PROCESSES WHEN

- Strong isolation / fault containment
- Security boundaries / different users
- Different memory / resources
- Stability more important than speed
- Untrusted code / 3rd-party plugins
- Language, separate runtime, crashes

11 PERFORMANCE TERMS BRIEFLY EXPLAINED

- **Context Switching** — Saving & restoring state of a running thread
- **Memory Usage** — RAM used by process / threads (including heap & stacks)
- **Cache Hits** — Data found in CPU cache (fast)
- **Synchronization Overhead** — Time spent in locks, messages, and coordination
- **Lock Contention** — Many threads wait for the same lock
- **Scheduler Overhead** — Work the OS does to schedule threads
- **Load Balancing** — Distributing work evenly across cores
- **CPU Affinity / Core Locality** — Keep thread on specific core (optional)

11 FINAL TAKEAWAYS

- Program = code on disk; Process = resource (address space, heap, files, FDs).
- Thread = execution path in a process; Lightweight; shares resources.
- Threads → great for performance; processes → great for isolation.
- Concurrency means overlapping progress. Parallelism means true simultaneous execution.
- Shared state + unsynchronized access = problems.
- Use the scheduler, run queues and cores: CPUs run one (or a few) threads at a time.
- Choose the right synchronization tool for safety and correctness.
- Design for simplicity first; avoid unnecessary shared state.
- Measure, profile, and test under load to understand real behavior.

12 FINAL TAKEAWAYS

- Program = code on disk; Process = resource (address space, heap, files, FDs).
- Thread = execution path in a process; Lightweight; shares resources.
- Threads → great for performance; processes → great for isolation.
- Concurrency = unoverlapping progress. Parallelism means true simultaneous execution.
- Shared state + unsynchronized access = problems.
- Use the scheduler, run queues and cores: CPUs run one (or a few) threads at a time.
- Choose the right synchronization tool for safety and correctness.
- Design for simplicity first; avoid unnecessary shared state.
- Measure, profile, and test under load to understand real behavior.